

MACHINE LEARNING ENGINEERING

FASE 5

AULA 7 - BOAS PRÁTICAS DE CÓDIGO PYTHON

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	13
REFERÊNCIAS.....	14

EMSE

O QUE VEM POR AÍ?

Nesta aula, vamos aprender sobre design patterns, SOLID e algumas configurações que podemos fazer para deixar o código limpo.

EMSE

HANDS ON

O conteúdo de SOLID dessa aula está no repositório da branch boas_praticas:
https://github.com/rocaabrera/curso_mlops/tree/boas_praticas.

Já o conteúdo onde aplicaremos as padronizações em nosso pacote estão na branch boas_praticas_empacotamento_codigo:

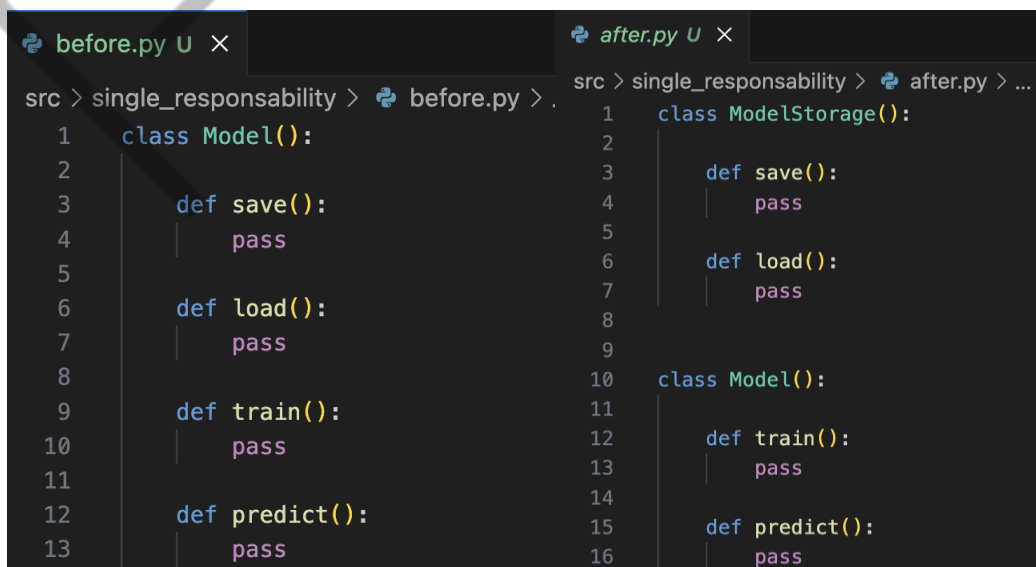
https://github.com/rocaabrera/curso_mlops/tree/boas_praticas_empacotamento_codigo.

SAIBA MAIS

Python é uma linguagem que nos permite escrever software utilizando orientação a objetos. Existem várias orientações para escrever o código de maneira orientada a objetos, e a mais famosa é a **SOLID**. Essa sigla é um acrônimo para os cinco princípios:

1. **Single Responsibility principle (SRP);**
2. **Open-closed principle (OCP);**
3. **Liskov substitution principle (LSP);**
4. **Interface segregation principle (ISP);**
5. **Dependency inversion principle (DIP).**

O princípio de **responsabilidade única** (single-responsability) diz basicamente para criar funções e classes que tem uma única responsabilidade. Ou seja, seu código deve fazer somente uma coisa dentro daquele escopo. Dessa forma, é possível aproveitar esse código em outros lugares. Lembre-se que quanto mais responsabilidades seu código tiver, mais específico para sua aplicação ele será. Esse princípio está relacionado ao conceito de **separation of concerns**, onde a ideia é que cada parte do código tenha uma seção específica para endereçar um único problema. A Figura 1 mostra um exemplo desse princípio:



```
before.py U X
src > single_responsability > before.py >
1 class Model():
2
3     def save():
4         pass
5
6     def load():
7         pass
8
9     def train():
10        pass
11
12    def predict():
13        pass

after.py U X
src > single_responsability > after.py > ...
1 class ModelStorage():
2
3     def save():
4         pass
5
6     def load():
7         pass
8
9
10 class Model():
11
12     def train():
13         pass
14
15     def predict():
16         pass
```

Figura 1 - Exemplo do princípio responsabilidade única
Fonte: Elaborado pelo autor (2024)

Perceba que, inicialmente, a classe **Model** tem a responsabilidade de realizar IO, pois tem os métodos **save** e **load**, porém a classe também é responsável por treinar (método **train**) e realizar a inferência (método **predict**). Podemos quebrar essa classe em duas preocupações diferentes, isto é, realizar tarefas relacionadas a IO (save e load) e tarefas relacionadas a CPU (train e predict). Você pode estar pensando: “deveriam ser quatro classes, uma para cada método”, mas isso vai depender de como você imagina o seu software. Minha sugestão é implementar o código de maneira evolutiva, começando por separar as responsabilidades, e conforme o código for crescendo, talvez seja necessário ter uma classe para carregar modelos clássicos e outra para carregar modelos baseados em redes neurais. Novamente, vai depender de cada caso.

O princípio de aberto-fechado diz que nosso código precisa ser fechado para modificações e aberto para extensão. Considere o código escrito na Figura 2:



```
before.py U X
src > open_closed > before.py > ...
1  from pathlib import Path
2  from typing import Any
3  import pickle
4
5  class ModelStorage():
6
7      def __init__(self, storage_type: str) -> None:
8
9          self.data = {}
10         self.storage_type = storage_type
11
12         def add(self, filename: str, model: Any) -> Any:
13             if self.storage_type == 'filesystem':
14                 try:
15                     path = (Path("tmp") / filename)
16                     path.parent.mkdir(parents=True, exist_ok=True)
17                     with path.open(mode="wb") as f:
18                         pickle.dump(model, f)
19
20                 except Exception as e:
21                     print(e)
22                     return None
23                 else:
24                     return str(path)
25
26             elif self.storage_type == 'inmemory':
27                 self.data[filename] = model
28
29             else:
30                 raise ValueError("Invalid storage type")
```

Figura 2 - Exemplo de código que não segue o princípio aberto-fechado
Fonte: Elaborado pelo autor (2024)

O problema deste código está no caso de precisarmos adicionar um novo tipo de armazenamento. Perceba que seria necessário alterar o método **add** (que já contém a maneira de adicionar modelos de outros tipos de armazenamento). Logo, nosso código estará mais complicado de estender e o seu funcionamento não está fechado para modificações, pois ao adicionar nova lógica, vamos alterar o funcionamento de um método já existente. Durante a construção do nosso pacote, o problema foi resolvido implementando o código da Figura 3:

```
after.py U X
src > open_closed > after.py > FileSystemModelStorage
1  import pickle
2  from typing import Any
3  from pathlib import Path
4  from abc import ABC, abstractmethod
5
6  class ModelStorage(ABC):
7      @abstractmethod
8      def add(self, filename: str, model: Any) -> str | None:
9          pass
10
11  class FileSystemModelStorage(ModelStorage):
12
13      def add(self, filename: Path, model: Any) -> str | None:
14          try:
15              path = (Path("tmp") / filename)
16              path.parent.mkdir(parents=True, exist_ok=True)
17              with path.open(mode="wb") as f:
18                  pickle.dump(model, f)
19
20          except Exception as e:
21              print(e)
22              return None
23          else:
24              return str(path)
25
26  class InMemoryModelStorage(ModelStorage):
27      def __init__(self):
28          self.data = {}
29
30      def add(self, filename: str, model: Any) -> str | None:
31          try:
32              self.data[filename] = model
33          except Exception as e:
34              print(e)
35              return None
36          else:
37              return filename
```

Figura 3 - Exemplo do princípio aberto-fechado implementado
Fonte: Elaborado pelo autor (2024)

Seguindo a nossa lista, temos agora o princípio da substituição de Liskov (**Liskov substitution principle**). Esse princípio diz que os Subtipos (InMemoryModelStorage e FileSystemModelStorage) devem ser substituíveis entre si em uma aplicação, ou seja, o programa que recebe objetos não precisa saber qual o tipo exato ele está recebendo, logo ele não precisa ser modificado para um tipo específico.

Em outras palavras, utilizamos ambos os subtipos pensando que eles são do tipo ModelStorage, mas para o programa (aplicação realizada na aula sobre containers), não importa qual subtipo estamos utilizando. Poderíamos argumentar que esses subtipos não são intercambiáveis, pois um demanda mais espaço em disco, enquanto o outro demanda mais memória. Entretanto, programar é entender os prós e contras das escolhas feitas, onde talvez o design escolhido não atenda a todos os princípios do SOLID.

O próximo princípio é o de segregação de interfaces, exemplificado na Figura 4:

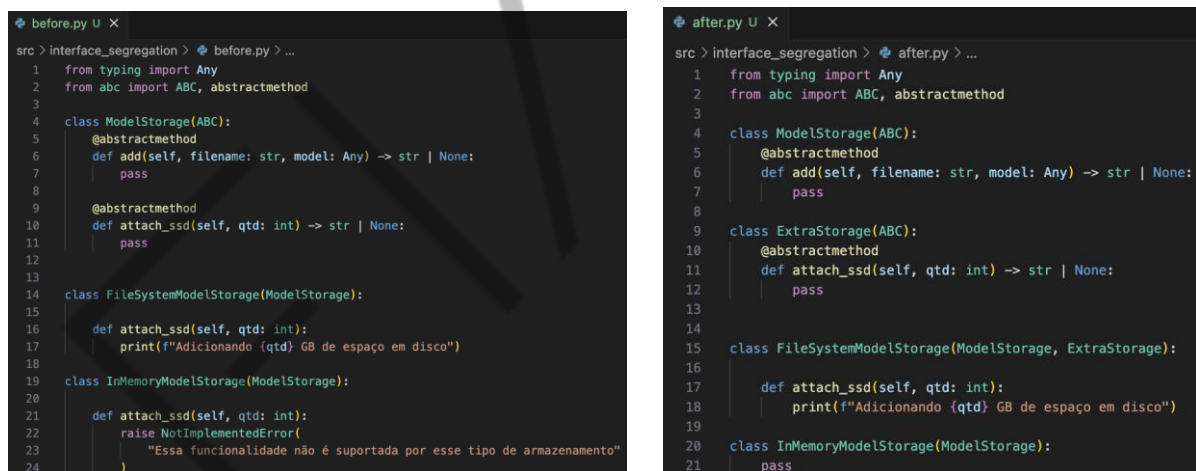


Figura 4 - Exemplo do princípio de segregação de interfaces

Fonte: Elaborado pelo autor (2024)

Imagina que você queira adicionar um método chamado **attach_ssd** na interface. Perceba que para não quebrar a interface, precisaremos implementar um método **attach_ssd** que não faz nada, conforme mostra a imagem esquerda na Figura 4. Esse método não faz sentido no contexto da classe InMemoryModelStorage, logo, nós precisamos quebrar essa interface conforme mostra a imagem direita na Figura 4. Porém, agora quebramos o princípio de Liskov. Uma alternativa seria adicionar um método chamado **increase_storage** na interface mas, mesmo assim, teríamos

difficuldade para implementar essa função no caso da classe `InMemoryModelStorage`, pois precisaríamos aumentar a quantidade de RAM disponível. Isso mostra que caso seja preciso adicionar essa funcionalidade, pode ser interessante rever a arquitetura do código como um todo.

O último princípio é o de inversão de dependência (**Dependency Inversion Principle**). Esse princípio diz o seguinte: “Abstrações não devem depender de detalhes. Detalhes devem depender de abstrações”. A aplicação docker desenvolvida segue esse princípio, uma vez que ela depende de uma abstração `ModelStorage`, e não depende de uma implementação específica, isto é, não depende de `InMemoryModelStorage` ou `FileSystemModelStorage`. Perceba a diferença desse princípio com o de Liskov, onde uma coisa é o código depender de uma abstração, e outra coisa é conseguir trocar duas implementações concretas de uma abstração de forma transparente.

Esse é um tema complicado e muitas vezes esses princípios conversam entre si e se misturam. A ideia desta aula é mostrar que existem orientações para construir um código mais fácil de manter e reutilizar.

Agora, vamos mudar um pouco de tema e conversar sobre algumas automatizações para melhorar o fluxo de desenvolvimento do nosso pacote. Inicialmente vamos criar um arquivo **test-requirements.txt** dentro do folder **tests** para colocar os pacotes necessários para executar os testes unitários do projeto. Em nosso caso, estamos utilizando o **pytest** para executar os testes e o **pytest-cov** para encontrar a cobertura dos nossos testes.

Agora, vamos padronizar o estilo em que o nosso código é escrito. Para fazer isso, vamos utilizar uma ferramenta de **lint**, a qual aplica uma análise estática do seu código. Existem várias, mas nesta aula vamos utilizar um **linter** escrito em Rust chamado [ruff](#). A Figura 5, mostra a execução do seguinte comando: **ruff check**. Este comando mostra várias melhorias que podemos implementar em nossa aplicação utilizando códigos padronizados previamente e, ao executar esse comando em nosso pacote, obteremos os seguintes códigos:

[F401](#): indica que existe um import no módulo que não está sendo utilizado.

[F403](#): indica que existe um import com wildcard (utilização do símbolo `*`) no módulo.

```

❖ (ins)$ ruff check
src/ml_communication/storage/__init__.py:1:1: F403 `from ml_communication.storage.data import *` used; unable to detect undefined names
1 | from ml_communication.storage.data import *
  | ~~~~~ F403
2 | from ml_communication.storage.model import FileSystemModelStorage, InMemoryModelStorage

src/ml_communication/storage/__init__.py:2:44: F401 `ml_communication.storage.model.FileSystemModelStorage` imported but unused; consider removing, adding to `__all__`, or using a redundant alias
1 | from ml_communication.storage.data import *
2 | from ml_communication.storage.model import FileSystemModelStorage, InMemoryModelStorage
  | ~~~~~ F401
= help: Use an explicit re-export: `FileSystemModelStorage as FileSystemModelStorage`

src/ml_communication/storage/__init__.py:2:68: F401 `ml_communication.storage.model.InMemoryModelStorage` imported but unused; consider removing, adding to `__all__`, or using a redundant alias
1 | from ml_communication.storage.data import *
2 | from ml_communication.storage.model import FileSystemModelStorage, InMemoryModelStorage
  | ~~~~~ F401
= help: Use an explicit re-export: `InMemoryModelStorage as InMemoryModelStorage`

tests/storage/test_model.py:3:21: F401 [*] `pathlib.Path` imported but unused
1 | import unittest
2 | from unittest.mock import MagicMock, patch
3 | from pathlib import Path
  | ~~~~~ F401
4 | from src.ml_communication.storage import InMemoryModelStorage, FileSystemModelStorage
  | ~~~~~
= help: Remove unused import: `pathlib.Path`

Found 4 errors.
[*] 1 fixable with the `--fix` option.

```

Figura 5 - Exemplo de execução do ruff para checar o pacote da aula
Fonte: Elaborado pelo autor (2024)

Nós podemos passar por todos os códigos indicados e resolvê-los de forma individual, mas o ruff indica que podemos resolver 1 dos 4 erros com a flag **--fix**, conforme mostra a Figura 5. A Figura 6 mostra o antes e depois da execução do comando.



```

❖ test_model.py X
tests > storage > test_model.py > ...
1 | import unittest
2 | from unittest.mock import MagicMock, patch
3 | from pathlib import Path
4 | from src.ml_communication.storage import InMemoryModelStorage, FileSystemModelStorage
5 |
6 | class TestModelStorage(unittest.TestCase):
7 |     def setUp(self):
8 |         mock = MagicMock()
9 |         mock.open.__enter__.return_value = True
10 |         self.model_path = mock
11 |         self.model = "dummy_model"
12 |
13 |     def test_in_memory_model_storage(self):

❖ test_model.py M X
tests > storage > test_model.py > ...
1 | import unittest
2 | from unittest.mock import MagicMock, patch
3 | from src.ml_communication.storage import InMemoryModelStorage, FileSystemModelStorage
4 |
5 | class TestModelStorage(unittest.TestCase):
6 |     def setUp(self):
7 |         mock = MagicMock()
8 |         mock.open.__enter__.return_value = True
9 |         self.model_path = mock
10 |         self.model = "dummy_model"

```

Figura 6 - Exemplo de execução do ruff para resolver os erros indicados no pacote da aula
Fonte: Elaborado pelo autor (2024)

Aplicando novamente o comando de check do ruff, temos somente três erros. Agora, vamos manualmente resolver esses três erros indicados, conforme mostra a Figura 7:

```
1 from ml_communication.storage.data import *
2 from ml_communication.storage.model import FileSystemModelStorage, InMemoryModelStorage

ml_communication > storage > __init__.py > ...
> from ml_communication.storage.model import FileSystemModelStorage, InMemoryModelStorage

__all__ = [FileSystemModelStorage, InMemoryModelStorage]
```

Figura 7 - Resolução manual dos erros apontados pelo ruff: imagem superior vem antes da correção e a imagem inferior vem após a correção
Fonte: Elaborado pelo autor (2024)

Além disso, existe o comando **ruff format** que permite resolver padronizações mais comuns, como por exemplo: se vamos utilizar tab ou espaço para indentação do código ou qual é o tamanho de linha máximo do projeto. Entretanto, em vez de aplicar esse comando com flags, podemos especificar essas padronizações utilizando o arquivo de configuração do pacote, o **pyproject.toml**.

```
pyproject.toml M X
pyproject.toml > ...
15 [tool.setuptools.dynamic]
21
22 [tool.ruff]
23 line-length = 100
24
25 [tool.ruff.format]
26 quote-style = "single"
27 indent-style = "tab"
28 docstring-code-format = true
```

Figura 8 - Especificação do que deve ser feito quando o comando **ruff format** for executado
Fonte: Elaborado pelo autor (2024)

Podemos fazer o mesmo para o comando **ruff check**, para, por exemplo, ignorar algum erro específico, conforme mostra a Figura 9:

```
pyproject.toml  ruff.toml

[tool.ruff.lint]
select = ["E", "F"]
ignore = ["F401"]
```

Figura 9 - Exemplo de configuração do Linter do ruff da [documentação base](#)
Fonte: Elaborado pelo autor (2024)

Agora, vamos automatizar o processo de linter utilizando hooks do git. Quando inicializamos um repositório, ele contém um diretório chamado **.git**, dentro dele existe

um diretório chamado **hooks**, o qual permite executar scripts entre as operações usuais do git, como por exemplo, um commit. Crie um arquivo no diretório de hooks com o nome **pre-commit**, e mude sua permissão para torná-lo executável (comando: **chmod +x pre-commit**). Dentro dele, adicione o comando ruff check.

```
● (ins)$ cat .git/hooks/pre-commit  
ruff check
```

Figura 10 - Exemplo ruff check em arquivo de pre-commit
Fonte: Elaborado pelo autor (2024)

Dessa forma, toda vez que você realizar um commit, ele só será aceito se você passá-lo comando ruff check. Perceba que você pode implementar o script bash que quiser no arquivo pre-commit, como por exemplo executar os testes unitários da sua aplicação. Antes de fazer o commit (**assista a videoaula para entender se essa realmente é uma boa ideia**), mude esse arquivo de acordo com o seu fluxo de trabalho. Vale comentar que existe um pacote Python chamado **pre-commit** que faz a mesma coisa e pode ser do seu interesse explorá-lo.

Não deixe de assistir a videoaula, pois vamos configurar uma forma de aumentar a segurança do seu repositório utilizando o pacote [bandit](#). Na videoaula, também comentaremos mais sobre hooks.

O QUE VOCÊ VIU NESTA AULA?

Nessa aula, introduzimos os conceitos de SOLID e falamos sobre a utilização de Git hooks. Vimos também sobre a padronização de código utilizando linters.

EMANDA

REFERÊNCIAS

GIT. Pro Git: customizando Git - **Git Hooks**. 2024. Disponível em: <https://git-scm.com/book/en/v2/Customizing-Git-Git-Hooks>. Acesso em: 10 set. 2024.

REAL PYTHON. **Solid Principles Python**. 2024. Disponível em: <https://realpython.com/solid-principles-python/>. Acesso em: 10 set. 2024.

RUFF. **Ferramenta Ruff: um linter e formatador de código Python extremamente rápido, escrito em Rust**. 2024. Disponível em: <https://github.com/astral-sh/ruff>. Acesso em: 10 set. 2024

PALAVRAS-CHAVE

Palavras-chave: SOLID. Padronização. Hooks.

EMPRE



POSTECH